BUY SOMETHING
Toade3

TECHNICAL BLUEPRINT | MOBILE APPLICATIONS

Technical Requirements Document

Toade3 Campus Logistics Platform

Customer, Vendor and Rider mobile applications

Supabase backend | React Native implementation proposed

DOCUMENT ID

TRD-TOADE3-2026-V1.1

VERSION

1.1.0 mobile revision

18 October 2026 | Draft for engineering and operations review

Campus operations, Ghana | Confidential

Document control

Field	Value
Title	Toade3 Campus Logistics Platform Technical Requirements Document
Document ID	TRD-TOADE3-2026-V1.1
Version and status	1.1.0 mobile application revision - draft for review
Date	18 October 2026
Source	Supplied Toade3 TRD v1.0; inherited parent reference SRS-TOADE3-2026-V2, version 2.0.0
Audience	Engineering, QA, DevOps, campus operations and product
Target architecture	Supabase backend with React Native mobile applications [PROPOSED implementation]
Deployment context	University campus operations in Ghana; bicycle courier fleet

Summary of the system

Toade3 connects customers who order food and items, vendors who prepare them, and riders who deliver by bicycle. All three user roles use installed mobile applications. Pricing, access control, dispatch and delivery verification remain server enforced.

At a glance	Mobile application baseline
Applications	Customer, Vendor and Rider; proposed shared React Native codebase with separate app binaries
Sign in	Ghana +233 SMS OTP and native Google sign in; iOS store requirement addressed in section 3.2
Pricing	GHS 5.00, 8.00 or 12.00 by zone; GHS 2.50 per km beyond 4.0 km, up to 5.0 km
Fleet limits	8.0 kg per trip; maximum two orders with pickups within 300 m; 5.0 km delivery radius
Realtime target	Vendor acceptance to an active customer screen in under 500 ms; push delivery measured separately
Approval checks	Original T1 to T4 plus mobile readiness checks M1 to M4

Revision basis
[MOBILE] identifies the requested change from browser portals to installed mobile apps. [SRS] references are inherited from v1.0; the separate SRS was not supplied for independent review. [PROPOSED] identifies engineering defaults and open decisions in section 19.

Contents

1 Introduction	4
2 System architecture	5
3 Authentication and identity	7
4 Authorization and row level security	8
5 Data architecture	10
6 Dynamic pricing engine	12
7 Order and delivery state machine	14
8 Dispatch engine and bicycle fleet rules	15
9 Realtime and native notifications	17
10 Interface specification	18
11 Mobile application architecture	19
12 Proof of delivery	21
13 Security architecture	22
14 Non functional requirements	23
15 Verification and test plan	24
16 Environments and delivery pipeline	26
17 Implementation roadmap	27
18 Requirements traceability	28
19 Assumptions and open decisions	29
Appendix A Reference database schema	31
Appendix B Technical reference sources	34

Mobile implementation choices are recorded in section 19. The reference schema and technical source links follow the main requirements.

1 Introduction

1.1 Purpose

This TRD translates the supplied Toade3 requirements into an engineering blueprint for mobile application delivery. It defines components, data, authorization, pricing, bicycle dispatch, native device features, performance targets and verification. Version 1.1 replaces the browser assumptions in version 1.0 throughout the document.

1.2 Scope

The product comprises Customer, Vendor and Rider mobile applications on one Supabase backend. It includes phone OTP and Google sign in, native navigation, database authorization, distance based pricing, realtime order tracking, push notifications, mobile offline caching, rider location reporting, bicycle routing and PIN or QR proof of delivery. [MOBILE]

Android and iOS are the proposed targets. The vendor app supports phone and tablet layouts. The proposed packaging keeps the three roles in separate signed apps while sharing domain and UI packages. The core product is not a browser app, PWA or website wrapped in a WebView. System authentication screens may be used only for identity verification.

Payment collection and settlement, refund processing and a customer facing admin application remain open scope items. Operators use a separately protected management process; no service credential is placed in any mobile app. The mobile mandate supersedes v1.0 statements that native applications are outside scope.

1.3 Requirement and proposal labels

[MOBILE] is the confirmed delivery change in this revision. [SRS] preserves business requirements as reported by v1.0. [PROPOSED] covers implementation, platform support, operational limits and additional acceptance gates that require product review. Section 19 records the decisions.

1.4 Glossary

Term	Meaning
RLS / JWT	PostgreSQL row level security / signed access token issued by Supabase Auth.
KOT / POD	Kitchen Order Ticket / proof of delivery using a customer PIN or QR token.
Native navigation	Screen stacks and tabs in the installed app, including guarded deep link destinations.
FCM / APNs	Firebase Cloud Messaging / Apple Push Notification service.
Foreground / background	App actively visible / app not visible and subject to operating system execution limits.
Haversine / GHS	Straight line great circle distance / Ghana cedi.
Edge Function / RPC	Server side Supabase function / callable PostgreSQL function.

2 System architecture

Three mobile clients communicate with Supabase over HTTPS and authenticated WebSocket connections. PostgreSQL is the system of record. Money, roles, fleet limits and state transitions are validated by server functions. Native device services provide secure storage, GPS, push handling and camera access.

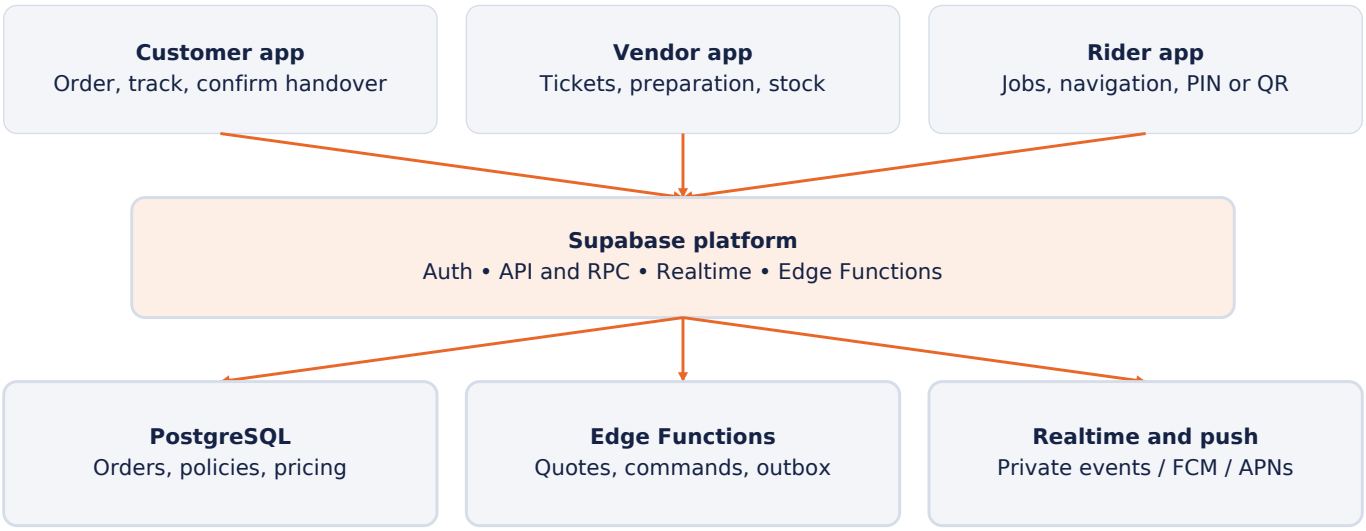


Figure 1. Mobile clients, shared backend and native delivery services. Push events are created by trusted server code after a committed business event.

2.1 Component responsibilities

Component	Responsibility	Trust
Customer app	Sign in, menus, cart, quote, ordering, order tracking and customer handover credential.	Untrusted client
Vendor app	Order tickets, foreground chime, native notifications, preparation actions and stock updates.	Untrusted client
Rider app	Eligible job feed, atomic claim, native navigation, trip location and delivery verification.	Untrusted client
Supabase Auth	OTP and federated identity verification; token issuance and session refresh.	Trusted platform
PostgreSQL	Data integrity, actor checks, RLS, transactions, fleet rules and immutable order snapshots.	Source of truth
Edge Functions	Quotes, repricing, order placement, proof verification, native device registration and push dispatch.	Trusted server
Realtime and push	Authorized foreground updates; minimal notification hints for backgrounded devices.	Transport only

2 System architecture continued

2.2 Architectural principles

- The server is the security boundary. A native screen guard prevents accidental access; database and function checks enforce permission. [SRS 3, 8; MOBILE]
- The server computes fees, distances, weights and legal transitions. Mobile estimates and cached data are never authoritative. [PROPOSED]
- Business writes use validated commands. Read APIs return role appropriate fields; proof secrets and internal money splits are not exposed by broad table reads. [PROPOSED]
- Design for weak mobile data: local read caches, compact events, foreground reconciliation and explicit offline states. [SRS 7; MOBILE]
- Background execution is conditional. Durable server state and push complement active realtime connections. [MOBILE]
- Fare and fleet configuration are versioned server data. App store releases are not needed to change a tariff. [PROPOSED]

2.3 Proposed technology stack

Layer	Mobile implementation
Application framework	React Native with Expo development builds, TypeScript, Metro and Hermes; compatible stable versions pinned at implementation.
Navigation and UI	React Navigation native stack and tabs; shared native components and Toade3 design tokens; no DOM dependent components.
Data and state	supabase-js; TanStack Query with a native persistence adapter; Zustand for small local UI state.
Local storage	SQLite for non-sensitive menus, receipts and route data; Keychain / Keystore backed adapter for session and delivery secrets.
Device features	Expo Notifications, Location, TaskManager, Camera and SecureStore; native build configuration and permission strings.
Maps and routing	Native map library compatible with selected SDK; versioned bicycle waypoint graph and on-device shortest path routing.
Backend	Supabase Auth, PostgreSQL, Realtime and Deno Edge Functions; Haversine distance in shared domain code and server validation.
Verification and delivery	Jest, React Native Testing Library, Maestro, pgTAP and k6; GitHub Actions; signed Android and iOS builds through EAS or native CI.

2.4 Brand tokens

Navy #152244 for app bars and headings; **secondary navy** #22346B for links; **orange** #E8682A for primary emphasis; **pressed orange** #C55029; **light orange** #FDEFE6 for ticket highlights. Check text contrast for each component and use navy or white text as appropriate. [Inherited design tokens]

3 Authentication and identity

Supabase Auth remains the identity store. Ghana phone OTP and Google sign in are the baseline methods; there is no email and password login. Every new account starts as a customer. Vendor and rider access requires operator approval. [SRS 2; MOBILE]

3.1 Phone OTP flow

1. Normalize the Ghana number to E.164 and validate its length and format. Example: 0241234567 becomes +233241234567. Use a numeric phone keypad and an explicit country code.
2. Call supabase.auth.signInWithOtp({ phone }). Display code entry and a resend timer. The SMS provider sends a six digit code.
3. Call verifyOtp({ phone, token, type: "sms" }). Allow manual entry and platform supported OTP autofill; do not request access to the SMS inbox.
4. Persist the session through the secure native storage adapter, fetch the profile and open the authorized native screen stack. A process restart must restore the session safely.

Proposed controls: OTP validity five minutes; one resend per 60 seconds; at most five requests per number and 20 per IP per hour; provider spend caps and spike alerts. Add a mobile compatible abuse challenge when risk requires it. Enforce limits on the server; a local timer alone is insufficient. [PROPOSED D-15]

3.2 Google sign in and iOS release condition

- Use the native Google sign in SDK to obtain an ID token, then exchange it with Supabase signInWithIdToken. Configure OAuth clients for each Android package, signing certificate and iOS bundle identifier. [PROPOSED]
- If a system authentication session is used as a fallback, use PKCE, state validation and an allowlisted mobile callback. Register environment specific app links / universal links and custom schemes; validate callback destination and reject replay. Do not embed the login page in a general WebView. [MOBILE]
- Complete callback handling for cold launch, background return, cancel, failure and expired sessions. Do not log callback tokens or authorization codes.
- For public iOS distribution with Google as a primary login, satisfy Apple guideline 4.8. The proposed solution is Sign in with Apple unless a documented exception applies. This changes the original two-method baseline and is a release decision D-18, not an assumed exemption. [R8]

3.3 Account creation and linking

The auth.users insert trigger creates public.profiles with role customer, never a role from user editable metadata. An approved, audited operator action performs vendor or rider upgrades. Duplicate phone and Google identities must not be merged solely by matching an unverified contact field. Identity linking is deferred unless explicitly enabled with reauthentication. [PROPOSED D-5, D-15]

> **Profile trigger invariant**
> The profile id equals auth.users.id. Role assignment is server controlled. Google sign in, phone OTP and any required Apple sign in all enter the same authorization model.

3 Sessions and 4 Authorization

3.4 Session and token handling

Item	Mobile requirement
Access token	Proposed lifetime one hour; SDK refresh while the app is active. A permitted rider background task refreshes explicitly when needed; do not assume a suspended timer runs.
Refresh token	Persist using an OS backed encrypted session adapter; support actual token sizes, rotation and concurrent refresh locking. Never use plain SQLite, logs or an unencrypted general preference store.
Native lifecycle	Register one AppState listener. Start foreground refresh on active; stop its timer outside active; on return refresh as needed and refetch authorized state. [R1]
Token claims	An optional trusted access token hook may add app_role. Sensitive commands verify the stored role and account status so a stale JWT cannot preserve an upgrade or revoked privilege.
Sign out	Revoke the chosen refresh sessions, remove this installation push registration, clear session and per-user caches, remove proof secrets and reset navigation. Stop rider tracking.
Revocation	An already issued access token can remain valid until expiry. Sensitive actions additionally check account / session status; never promise that deleting a local token instantly revokes all API access.

4 Authorization and row level security

Native navigation and server authorization provide separate controls. An unauthorized role must never mount a privileged screen or fetch its data. Server functions and RLS still reject a modified app or a forged request. [SRS 3, 8; MOBILE]

4.1 Native screen guard

1. Hold the splash screen while the stored session is loaded. A missing or invalid session opens the SignIn stack.
2. Fetch the authoritative profile. Mount only the screen stack allowed for this app and role. An approved vendor in the customer app is directed to the appropriate app / support flow; it is never granted extra screens.
3. Treat deep links and notification taps as untrusted navigation requests. Load session and role first, check order ownership on the server, then open the destination.
4. Reset the navigation stack and clear query caches on sign out, account switch or role change. Hardware back and restored navigation state cannot reopen protected screens.

```
const { session, profile, loading } = useSession();
if (loading) return <SplashScreen />;
if (!session) return <SignInStack />;
if (!profile || profile.role !== EXPECTED_APP_ROLE)
  return <WrongAppScreen />;
return <AuthorizedAppStack />;
```

Listing 1. Native guard pattern. Screen names are navigation identifiers; no React Router redirects or browser paths are required.

4 Authorization continued

4.2 Access matrix

Resource or action	Customer	Vendor	Rider	Anonymous
Profile	Own row; safe fields only	Own row; safe fields only	Own row; safe fields only	None
Vendor and menu reads	Public catalog	Own kitchen and catalog	Assigned pickup details	Public catalog
Menu writes	None	Own vendor only	None	None
Order reads	Own orders	Own kitchen orders	Limited feed; assigned job detail	None
Create order	place-order only	None	None	None
Order transition	Cancel own eligible order	Own kitchen states	Delivery command only	None
Delivery reads	Own order status	Own kitchen order status	Own assignments	None
Delivery writes	None	None	Assigned rider command	None
Location reads	Active assigned rider only	No continuous tracking	Own telemetry	None
Device registration	Own installation	Own installation	Own installation	None

4.3 Policy and API patterns

Enable RLS on all exposed tables and grant only the required operations. Keep helper functions in a non-exposed private schema. Privileged RPCs require an explicit authenticated actor, stored role, ownership check, controlled search_path and narrow EXECUTE grants. Do not trust user_metadata for authorization.

```
alter table public.profiles enable row level security;
create policy profiles_select_own on public.profiles
  for select to authenticated using (id = (select auth.uid()));
create policy profiles_update_own on public.profiles
  for update to authenticated
  using (id = (select auth.uid()))
  with check (id = (select auth.uid()));
revoke update on public.profiles from authenticated;
grant update (full_name) on public.profiles to authenticated;
```

Listing 2. Representative ownership and column protection. Role, phone and email changes use validated identity / operator workflows; a profile update cannot elevate the role.

Limit the rider feed at the server
Unclaimed orders are exposed only by a role checked, field limited rider_feed RPC. Grant no broad rider SELECT policy on unclaimed base order rows. Reveal customer contact details only after a successful claim, and remove access when the active job ends.

4.4 Unauthorized screen and API behavior

A customer opening a VendorOrders screen is redirected within native navigation before any vendor data loads. vendor_orders raises SQLSTATE 42501 / HTTP 403 for the wrong role. An RLS filtered table read may return an empty set rather than 403; tests must distinguish these outcomes. [PROPOSED]

5 Data architecture

The five inherited core entities remain profiles, vendors, menu_items, orders and deliveries. Supporting entities enforce weights, item snapshots, drop points, configuration and telemetry. Native push adds device registration and a server controlled notification outbox. [SRS 5; PROPOSED additions]

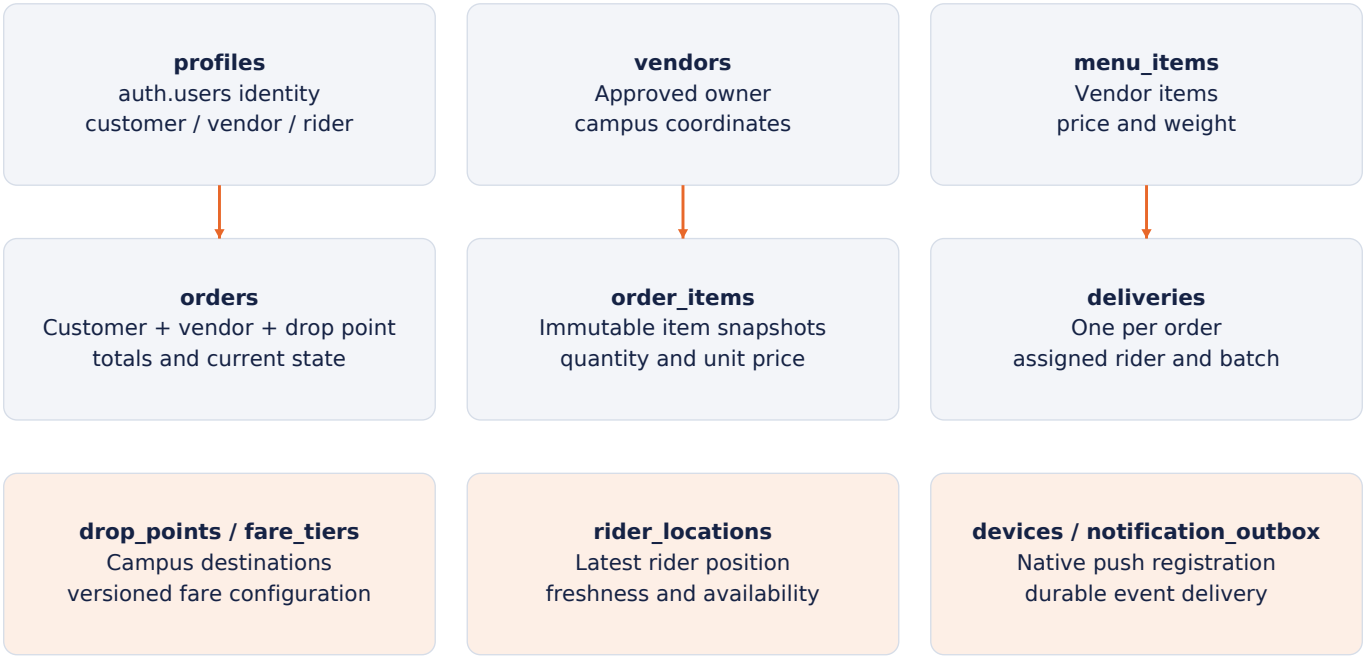


Figure 2. Core entities and supporting mobile data. Orders reference customer, vendor and drop point; order_items belong to an order; each order has at most one delivery. Rider assignment references a profile.

5.1 Data boundaries

- The server stores canonical orders and timestamps. Mobile SQLite contains a disposable read cache, not an independent order ledger.
- Snapshot menu title, unit price and item weight on order placement. Later menu edits cannot change historical totals or payload weight.
- Keep handover hashes, failed-attempt state, QR nonces, idempotency records and notification jobs inaccessible to general client reads. Role-specific read functions return only authorized columns.
- Device tokens belong to a user plus app installation and environment. They are not identity proof. A fresh login rebinds the installation, and sign out revokes its association.
- Rider telemetry records device observation time, server receipt time and accuracy. Customer tracking ends on completion or cancellation; precise location is never broadcast to a campus-wide feed.
- The supplied source does not define campus tenant identifiers. A single-campus pilot is the default; multiple campuses require campus_id on scoped data, tenant-aware RLS and channel membership before rollout. [PROPOSED D-21]

Reference schema status
Appendix A preserves the core data structure and identifies mobile additions. It is an engineering reference, not a complete production migration. Deployable migrations must include all policies, grants, triggers, command functions and seed configuration, with the tests in section 15.

5 Data definitions continued

5.2 Tables and constraints

Table	Essential fields and rules
profiles	id references auth.users; role defaults to customer; full_name; verified contact fields; created_at and updated_at. Role changes are operator controlled.
vendors / menu_items	One approved owner per vendor; valid lat/lng; vendor opening state. Each item has non-negative numeric price, positive known weight_g and availability.
orders	Customer, vendor, rider and drop point references; current status; priced distance; item total, fee, commission, payout, grand total and total_weight_g; server timestamps.
order_items	Order and menu item references; immutable title, unit price and weight snapshots; qty must be positive.
deliveries	Unique order_id; assigned rider; assigned / picked_up / delivered; batch_id and lifecycle timestamps.
drop_points / fare_tiers	Active campus drop points and coordinates. Versioned fare bands, base fee, commission method, per-km extension, SLA and effective dates.
rider_locations	One latest position per rider; lat/lng, accuracy_m, observed_at, received_at and availability / heartbeat fields. Apply coordinate and freshness validation.
devices [MOBILE]	Owner, installation_id, app_role, platform, environment, token, enabled flag and last_seen_at. Installation and token uniqueness enforced.
notification_outbox [MOBILE]	Committed event id, recipient, minimal payload, delivery status, attempt count and next_attempt_at; server only.

5.3 Enumerations

Enum	Values
user_role	customer, vendor, rider; no end-user admin role by default.
order_status	pending, accepted, preparing, ready, out_for_delivery, delivered, cancelled, rejected.
delivery_status	assigned, picked_up, delivered. Cancellation of an unpicked assignment uses the cleanup workflow in section 7.

5.4 Indexes and integrity rules

- Index orders by (vendor_id, status, created_at), (customer_id, created_at), and the unclaimed accepted / preparing / ready queue; index deliveries by (rider_id, status).
- Use numeric money values or integer pesewas for calculation. Enforce total_amount = items_total + delivery_fee and delivery_fee = commission + rider_payout.
- Unique idempotency keys are scoped to actor and operation. Use monotonic row versions for reconciliation; timestamps remain audit fields.
- Do not treat unknown weight as zero. Items without verified positive weight cannot be ordered until their payload value is supplied. [PROPOSED D-6]

6 Dynamic pricing engine

The mobile checkout requests an authoritative quote using the vendor and selected campus drop point. An Edge Function calculates Haversine distance and applies the inherited fare matrix. The app displays the result; it cannot submit an authoritative fee. [SRS 4, 4.1; MOBILE]

6.1 Fare configuration

Zone	Priced distance	Fee GHS	Commission	Rider payout	Target time
Inner Quad	0.0 to 1.0 km	5.00	1.00	4.00	10-15 min
Mid Campus	1.1 to 2.5 km	8.00	1.50	6.50	15-25 min
Outer Perimeter	2.6 to 4.0 km	12.00	2.00	10.00	25-35 min
Extended Zone	Over 4.0 to 5.0 km	12.00 + 2.50 × excess km	15% of fee	Fee minus commission	35+ min

Configuration changes are versioned. Store the applied fare version and price snapshot with the order so a later change does not rewrite history. The extended charge applies only to distance beyond 4.0 km. [PROPOSED D-1]

6.2 Distance calculation

```
const EARTH_RADIUS_KM = 6371.0088;
const rad = (degrees: number) => degrees * Math.PI / 180;
function haversineKm(a: Point, b: Point): number {
  const dLat = rad(b.lat - a.lat), dLng = rad(b.lng - a.lng);
  const x = Math.sin(dLat / 2) ** 2
    + Math.cos(rad(a.lat)) * Math.cos(rad(b.lat))
    * Math.sin(dLng / 2) ** 2;
  return 2 * EARTH_RADIUS_KM * Math.asin(Math.sqrt(Math.min(1,x)));
}
```

Listing 3. Haversine uses vendor-to-drop-point straight line distance. Bicycle route length may be longer; route geometry does not silently change the inherited pricing basis.

6.3 Server fee algorithm

1. Validate coordinates and active drop point. Calculate distance, then round up to the next 0.1 km to close the original gaps between bands. [PROPOSED D-3]
2. Reject a priced distance above 5.0 km. Apply fixed fees through 4.0 km; beyond that use fee = 12.00 + 2.50 × (priced_km - 4.0).
3. Calculate with integer pesewas or a decimal money library, using half-up rounding to two decimals. Extended commission is 15% of the fee; rider payout is fee minus rounded commission. [PROPOSED D-8]
4. Read current menu prices and weights from the server. Calculate items_total, total_weight_g and total_amount; reject unavailable items and a single order over 8,000 g.
5. Return an expiring quote. On place-order, re-price and revalidate in the write path; require customer reconfirmation if the amount changes. Persist one immutable snapshot.

6 Pricing verification and quote interface

6.4 Pricing test vectors

Raw km	Priced km	Zone	Fee GHS	Commission	Rider payout
0.40	0.4	Inner Quad	5.00	1.00	4.00
1.00	1.0	Inner Quad	5.00	1.00	4.00
1.04	1.1	Mid Campus	8.00	1.50	6.50
2.00	**2.0**	**Mid Campus**	**8.00**	**1.50**	**6.50**
2.50	2.5	Mid Campus	8.00	1.50	6.50
2.55	2.6	Outer Perimeter	12.00	2.00	10.00
4.00	4.0	Outer Perimeter	12.00	2.00	10.00
4.10	4.1	Extended	12.25	1.84	10.41
4.50	4.5	Extended	13.25	1.99	11.26
5.00	5.0	Extended	14.50	2.18	12.32
5.01	5.1	Rejected	OUT_OF_RADIUS	-	-

The 2.0 km case preserves the inherited acceptance requirement exactly. The fare table governs the first three zones; the original generic formula does not define the needed parameters for those zones. [PROPOSED D-2]

6.5 quote-delivery Edge Function

Contract	Specification
Endpoint	POST /functions/v1/quote-delivery; authenticated customer only.
Request	vendor_id, drop_point_id, items [{ menu_item_id, qty }]. Do not accept fee, item price, commission or payout as input authority.
Success	distance_km, zone, delivery_fee, items_total, total_amount, total_weight_g, eta_min, quote_id, fare_version and expires_at.
Errors	400 invalid input; 401 no valid session; 403 wrong role; 409 item unavailable; 422 OUT_OF_RADIUS or OVER_WEIGHT.
Placement	place-order accepts quote_id and a persistent Idempotency-Key. A changed price, expired quote or unavailable item requires a new quote and explicit confirmation.
Visibility	Customer sees item total, distance, zone, fee and grand total. Commission and rider payout are role restricted server fields, not hidden only by the UI.

Mobile retry safety
Disable duplicate taps while a request is pending. If the response is lost, query the operation result or retry the same idempotency key. Never create a new key just because connectivity failed. Orders cannot be queued for automatic offline submission.

7 Order and delivery state machine

The vendor advances kitchen states. The assigned rider advances delivery states. The database validates actors and transitions and updates the two state machines within one transaction. The app shows a pending action until server acknowledgement; it never treats an offline tap as a committed status. [SRS 5; MOBILE]



Delivery lane: assigned → picked_up → delivered; final two advance the order.

Figure 3. Main order path. A pending order may be rejected. A customer may cancel pending or accepted orders before preparation, subject to the rules below.

7.1 Allowed order transitions

From	To	Actor	Guard
pending	accepted	Own vendor	Vendor open; items available.
pending	rejected	Own vendor	Reason code required. [PROPOSED]
pending or accepted	cancelled	Order customer	Preparation not started; clean up any unpicked assignment.
accepted	preparing	Own vendor	Valid current state.
preparing	ready	Own vendor	Valid current state.
ready	out_for_delivery	System	Assigned rider pickup accepted.
out_for_delivery	delivered	System	Assigned rider proof verified on server.

7.2 Delivery transitions

Transition	Actor	Rule
None to assigned	Rider via claim_job	Accepted / preparing / ready order; all fleet checks pass; unique assignment.
assigned to picked_up	Assigned rider	Order must be ready; stamp picked_up_at.
picked_up to delivered	Assigned rider	Valid customer PIN or QR; stamp delivered_at once.

Cancellation locks the order and related unpicked delivery, releases its assignment and recalculates the rider batch atomically. A proposed implementation archives then deletes the unpicked assignment while retaining an audit event. No cancellation may leave a live job in a rider queue. Refunds and exceptional post-pickup cancellations remain D-7.

7 Enforcement and 8 Dispatch

7.3 Transaction and command enforcement

```
advance_order(order_id, next_status, expected_version):
  require authenticated actor with approved vendor role
  lock order; verify actor owns its kitchen
  verify current version and permitted transition pair
  validate item / opening conditions for acceptance
  update status, version and server updated_at
  write audit event and notification outbox event
  commit; return authoritative role-safe order detail
```

Listing 4. Command pattern. Repeated taps, another device and a resumed app all use the same server checks. Reject stale state with a conflict response and refresh the ticket.

8 Dispatch engine and bicycle fleet rules

Rule	Limit	Enforcement	Failure
Payload weight	8.0 kg per trip	place-order and claim_job across active batch	OVER_WEIGHT
Batch size	At most two orders	claim_job in one transaction	BATCH_FULL
Pickup separation	At most 300 m	Second claim against first active vendor	PICKUPS_TOO_FAR
Delivery radius	At most 5.0 km	Quote, placement and claim validation	OUT_OF_RADIUS

8.1 Atomic job claim algorithm

1. The Rider app sends claim_job(order_id, idempotency_key). Verify approved rider status, availability and a sufficiently fresh server-held location.
2. Acquire a transaction-scoped per-rider lock, then lock the target order. This serializes claims for the same rider across devices as well as claims for one order across riders.
3. Require rider_id to be null and the order state to be accepted, preparing or ready. If another rider already claimed it, return JOB_TAKEN.
4. Load and lock the rider active assignments. Reject a third job, total payload above 8,000 g or a second pickup more than 300 m from the first.
5. Recheck order radius and operating eligibility using server coordinates. Do not use an untrusted submitted position to change fare or relax fleet rules.
6. Create one delivery, assign orders.rider_id and reuse the batch_id when stacking. Commit atomically and return only authorized job details. Retries return the same result.

Concurrency acceptance
Test simultaneous claims by different riders and simultaneous claims for different orders by the same rider. Exactly one rider may own a job and no concurrent requests may bypass the two-order or 8.0 kg limits.

8 Rider location and navigation

8.2 Geofenced job feed

- rider_feed uses the authenticated rider and recent server-held coordinates, not arbitrary query coordinates. Filter pickups within 5.0 km, then sort by distance to pickup. [PROPOSED]
- Each job card shows pickup distance, payout, drop-off and payload weight. Jobs that fail fleet limits are disabled with the reason. No customer name, phone or proof credential appears before assignment. [SRS 7.3]
- While the app is active, refetch after a job event and every 30 seconds as a safety net. Recheck all eligibility during claim; a displayed card is not a reservation.

8.3 Native location lifecycle [MOBILE]

Rider state	Location behavior
Offline	Stop location tasks and reporting. Do not collect location from a signed-out rider.
Online and available	Foreground target: one update every 10 seconds, adjusted for movement. Background availability uses a declared permitted location service or becomes unavailable when heartbeat expires.
Active delivery	Foreground target: one update every five seconds. Continue a native background task during the trip when permission and OS execution allow; intervals are targets, not guarantees.
Background setup	Configure the appropriate Android foreground location service and persistent indicator, and iOS location background mode. Request additional background permission only when the chosen execution model needs it. [R2, R7]
Stale or denied	If location / heartbeat is older than 90 seconds, block new claims and show last known time. Keep active job and support access available; mark tracking stale rather than inventing movement. [PROPOSED]
Completion / force stop	Stop trip tracking after the last active job. OS termination or force stop can interrupt delivery updates; reconcile on next launch and alert operations if an active trip becomes stale.

Record accuracy and observation time, reject invalid coordinates and flag implausible jumps. Upload only recent required telemetry; do not replay a long stale trail as live tracking. A background task obtains a valid session before sending data. [PROPOSED D-16]

8.4 Native bicycle navigation

Bundle a versioned campus waypoint graph with bicycle-permitted nodes and edges. Compute the route on device, draw it with a native map component, and show textual guidance to pickup and drop-off. Cache the graph and permitted map resources; map tiles are subject to provider licensing. If tiles are unavailable, textual waypoints and destination coordinates remain usable. [PROPOSED]

> **Permissions are tied to the role**
> Customers may select a campus drop point without GPS. Vendors need no continuous location access. Riders receive a clear explanation when going online or beginning a trip; permission denial leads to a visible recovery or foreground-only flow, never a silent assumption of background coverage.

9 Realtime and native notifications

Foreground order updates use authenticated Supabase Realtime channels. When the app is inactive, native push may alert the user and lead back to a fresh server read. Neither a persistent background WebSocket nor instant push delivery is guaranteed. [SRS 9.4; MOBILE]

9.1 Channel and event design

Recipient	Authorized scope	Event purpose
Customer	Private user / order channel after ownership check	Order accepted, preparation, pickup and delivery; no internal money split or PIN.
Vendor	Private own-kitchen channel	New ticket, cancellation and current state invalidation.
Rider	Private campus rider feed and own-job channels	Job availability / removal hints and assignment state; contact detail only via assigned-job read API.
Tracking customer	Private channel for active own order	Latest assigned-rider position with freshness; revoke on job completion.
Catalog	Public-safe menu availability data	Invalidate affected vendor menu; Postgres Changes may be used only for safe exposed fields.

Use server-originated minimal Broadcast payloads for sensitive order entities, followed by role checked reads. Channel filters and client-supplied topic names are not authorization. Test private channel membership and avoid broadcasting a full base order row. [PROPOSED]

9.2 Foreground latency budget

The inherited 500 ms target applies to connected, active vendor and customer apps. Proposed allocation: tap to API 60 ms; database 40 ms; fan-out 60 ms; mobile network 150 ms; native render 50 ms; headroom 140 ms. These are design budgets to measure, not achieved results. T4 reports the network profile and p95.

9.3 Reliability and reconciliation

- Subscribe and buffer new events while fetching a versioned snapshot, then apply only newer events. Refetch on every foreground return, reconnect and notification tap to close missed-event gaps.
- Use reconnect backoff of 1, 2, 4, 8 and at most 15 seconds while active; show reconnecting / stale state. An event id and row version deduplicate out-of-order or repeated events.
- A disconnected vendor app warns after 20 seconds. A stale vendor heartbeat stops automatic acceptance assumptions and triggers an operational missed-order escalation.

9.4 Native push and vendor alerts

A committed event writes a durable outbox entry. A trusted worker sends through Expo Push Service to FCM / APNs, processes tickets and receipts, retries transient failures and disables invalid device tokens. Send event id, resource id and a generic message; never send PINs or private contacts. [R3, R4]

Start shift checks notification and sound settings. While active, show a light-orange ticket and play a chime every 20 seconds until acknowledgement. In background, use OS notification channels / categories and allowed sound; silent mode, user settings and OS limits may suppress it. Repeated alerts and escalation are server controlled; push is not a real-time SLA.

10 Interface specification

Mobile apps use HTTPS functions / RPCs with validated JWTs and role checks. Table reads, where granted, remain under RLS. Critical writes are acknowledged only after a committed server transaction. Errors have stable codes that the native apps translate into clear messages.

10.1 Callable interfaces

Interface	Caller	Input and effect
quote-delivery [Edge]	Customer	Vendor, drop point and items → validated expiring fee and weight quote.
place-order [Edge]	Customer	quote_id + Idempotency-Key → one order and customer handover credential.
get_order [RPC]	Owner / own vendor / assigned rider	order_id → role-shaped order and delivery detail; secret columns excluded.
cancel_order [RPC]	Customer	order_id, expected_version → cancel if allowed; release any unpicked assignment.
advance_order [RPC]	Vendor	order_id, next_status, expected_version → validated kitchen state.
set_item_availability [RPC]	Vendor	item_id, availability → own vendor item only.
vendor_orders [RPC]	Vendor	Status filter and cursor → own kitchen tickets; other roles receive 403.
rider_feed [RPC]	Rider	Uses stored current location → limited eligible job cards.
claim_job [RPC]	Rider	order_id + idempotency key → atomic assignment or fleet conflict.
update_delivery [RPC]	Assigned rider	delivery_id, picked_up, expected_version → pickup only after ready.
verify-handover [Edge]	Assigned rider	order_id + PIN or QR + idempotency key → verify and complete atomically.
update_rider_location [RPC]	Rider	Coordinates, accuracy, observation time → validated own telemetry.
register-device / unregister-device	Signed-in user	Installation, app, platform and push token → own registration / revocation.
get_handover [Edge]	Order customer	Reauthenticated access → reissued credential if recovery is needed; old credential invalidated.
request-account-deletion [Edge]	Signed-in user	Reauthenticated request → tracked deletion / anonymization workflow.
admin_set_role [server only]	Approved operator	User, role and approval reference → audited upgrade; not mobile callable.

10.2 Error codes

401 SESSION_EXPIRED; **403** FORBIDDEN; **409** JOB_TAKEN, BATCH_FULL, PICKUPS_TOO_FAR, ITEM_UNAVAILABLE, STALE_VERSION or QUOTE_CHANGED; **422** OUT_OF_RADIUS, OVER_WEIGHT, ILLEGAL_TRANSITION or PIN_INVALID; **429** PIN_LOCKED or RATE_LIMITED. A denied or stale device location uses LOCATION_REQUIRED / LOCATION_STALE with a recovery instruction.

Retries and response loss
Persist the pending operation id before sending a critical command. On an ambiguous result, recover by operation id / idempotency key and refetch state. Do not silently replay cancellation, pickup, acceptance or handover after an offline interval.

11 Mobile application architecture

Three separately signed mobile apps share native packages in one monorepo. Role separation remains an authorization rule at the server even when binaries are separate. Metro produces Android and iOS bundles; there is no Vite web build or browser route layer. [MOBILE; packaging PROPOSED]

```
toade3/
  apps/
    customer/   vendor/   rider/   # Expo / React Native apps
  packages/
    ui/          # native controls, tokens, tickets, status chips
    auth/        # native sign in, secure session, role guard
    api/         # typed Supabase client, DTOs and query hooks
    domain/      # states, fare types, Haversine, money rules
    realtime/    # subscriptions, snapshots, event de-duplication
    storage/     # SQLite caches and secure storage adapters
    device/      # push, location, camera, app link handlers
  supabase/
    migrations/  functions/  seed.sql  tests/
  e2e/           # Maestro flows and shared device fixtures
```

Listing 5. Proposed native workspace layout. Share domain logic and design tokens; do not carry over HTML elements, CSS-only widgets, DOM APIs or browser storage.

11.1 Customer application [SRS 7.1; MOBILE]

- Native Phone / Google sign in choices, country code hint, numeric OTP entry and supported autofill. Apply the iOS release decision in section 3.2.
- Vendor list, live availability, menu, cart, campus drop-point picker and itemized distance / zone / fee / total panel. Debounce quote requests by 400 ms after cart or drop-point changes.
- Native tracking screen with status steps, assigned rider name, ETA and location freshness. Delivery updates also appear as permission-based push notifications.
- Menus and redacted order receipts load from a per-user cache, show their age and refresh when connected. Checkout always requires a fresh server quote.

11.2 Vendor application [SRS 7.2; MOBILE]

- On phones, use status tabs and a ticket list; on tablets, use a multi-column KOT board. Avoid desktop hover interactions and horizontal overflow.
- Show one kitchen action at a time: Accept, In Preparation, Ready for Pickup. Keep the ticket pending until the server confirms; recover stale / failed actions explicitly.
- Use native switches for stock. Start shift checks sound, notifications and connection; an explicit foreground kitchen mode may keep the display awake while visible.

11.3 Rider application [SRS 7.3; MOBILE]

Show geofenced jobs with payout, drop-off and weight; one-tap server claim; active batch; native map and route guidance; trip location status; and PIN entry / QR camera scan. A prominent online switch and tracking indicator show whether new jobs and background tracking are available.

11 Native lifecycle and offline behavior

11.4 Device permissions

Permission	When requested	Denied or unavailable
Notifications	After sign in with a role-specific explanation	Core screens remain usable; show settings guidance. Vendor operations use foreground monitoring / escalation until alerts are enabled.
Customer location	Only when customer taps Use my location	Allow manual selection of a campus drop point.
Rider location	When going online; background access when required for an active trip	No new location-dependent claim without a valid fix; preserve active job access and show support / recovery.
Camera	When rider chooses QR handover	Offer manual customer PIN entry.
Vendor media picker	When adding / changing a menu image	Use the OS picker; keep text and stock actions available.

Do not ask for microphone, contacts or unrestricted SMS access for these workflows. Explain the purpose before a prompt; if permanently denied, open the relevant system settings screen and reconcile when the app returns. [PROPOSED]

11.5 App lifecycle

- Cold launch: restore secure session, load the profile, select the permitted stack, then reconcile current orders before acting on a notification / deep link.
- Background: stop foreground-only timers and subscriptions as appropriate. Rider location continues only through the configured permitted native service; business state remains on the server.
- Foreground return: verify session, refresh current role and order state, renew subscriptions, process a safe pending destination and refresh push registration if the token changed.
- Process loss or update: restore only durable operation ids and safe drafts; query whether a submitted command committed. Never infer delivery success from a previous local button state.

11.6 Offline storage and synchronization

Data or action	Offline rule
Menus / campus route graph	SQLite cache with version and fetched_at. Clearly label cached availability.
Cart draft / redacted receipts	Per-user cache. Revalidate every item and quote before checkout; clear on account switch.
Tokens / proof credentials	Encrypted native storage adapter only; no PIN, refresh token or private contact in plain SQLite, push payloads or crash logs.
Place / claim / change state / handover	Connection required. Do not show success or queue a new irreversible command while offline.
Pending ambiguous command	Keep operation id, then resolve its server outcome after reconnect before offering a retry.
Rider telemetry	Buffer a small bounded recent window if permitted; discard expired samples and label stale customer tracking.

Storage is a cache, not a backup. Apply per-user isolation, expiry, platform backup exclusions and sign-out cleanup. Full menu image / tile downloads should be bounded and user visible. [PROPOSED D-17]

12 Proof of delivery

An order is delivered only after the assigned rider submits a valid customer PIN or QR token and the server commits completion. Camera access is a native permission, and a manual PIN remains available. Offline scanning or local validation alone cannot complete a delivery. [SRS 7.3; MOBILE]

12.1 PIN method

1. place-order generates a cryptographically random four digit PIN. Store a protected verifier, never a readable PIN, in a non-exposed server store. Return the credential only to the customer.

2. Persist the customer credential with the secure native storage adapter so a process restart does not lose it. Do not put it in a notification, general receipt cache, analytics event, screenshot preview or rider read API.

3. At handover, the customer reads the PIN to the rider. The rider sends it with order id and idempotency key to verify-handover over an authenticated connection.

4. The server checks actor, assignment, picked_up state and lockout, then compares the verifier in constant time. After five wrong attempts, lock the order for 10 minutes and notify operations. [PROPOSED]

5. A valid proof atomically completes the delivery and order, consumes the credential and records delivered_at. A repeated successful request returns the same result rather than creating another event.

12.2 QR method

- The customer app receives a server-signed token containing order id, a random nonce and expiry. The signing key remains in server secrets.
- Generate / refresh the QR after pickup. Its proposed lifetime is 30 minutes from pickup; a customer-only recovery flow can reissue an expired credential while invalidating its predecessor.
- The Rider app opens a native camera scanner only on request. Reject malformed or unrelated QR content. Submit the token to the same server completion path; verify signature, expiry, assignment and unused nonce.
- If the camera is denied, damaged or unavailable, switch to PIN entry. If connectivity is lost, show that verification is pending and use the support process; never show Delivered until committed.

12.3 Recovery and protection

A four digit PIN has only 10,000 possibilities, so salted hashing alone is insufficient against an offline attacker. Combine a protected verifier with a server-held secret, strict attempt limits and isolated storage. Clear proof credentials after completion, cancellation or sign out. [PROPOSED]

> **Lost device or credential**
> The customer may reauthenticate and request a new credential for an active own order. Reissue invalidates the old PIN / nonce and is rate limited and audited. The assigned rider cannot use the recovery endpoint to obtain the customer secret.

13 Security architecture

Every mobile request is untrusted. RLS, explicit API grants and validated functions protect server data; platform security protects local sessions and permissions. Code signing does not replace authorization. [SRS 8; MOBILE]

13.1 Threats and controls

Threat	Control
Role escalation	Never assign role from user metadata. Restrict profile column updates and operator role commands; audit changes.
Other users data / IDOR	Authorize every resource id. Field-limit the rider feed and role-shaped order APIs; test all ownership combinations.
Fee or state tampering	Recompute money and payload on server; use row locks, expected versions and permitted transitions.
OTP abuse	Server-enforced number / IP limits, risk challenges, spend caps and failure monitoring.
Proof brute force / replay	Protected verifier, five-attempt lockout, expiring one-use QR, actor checks and idempotent completion.
Location spoof / stale tracking	Validate coordinates, freshness and accuracy. Flag implausible travel; server vendor/drop-point coordinates enforce fare and fleet rules.
Token or service key leakage	OS backed encrypted session storage. Only publishable project key in the binary; service and push-provider secrets remain server side.
Unsafe native links / input	Allowlist deep links, validate state and parameters, authorize after navigation; never execute QR or notification content as code.
Local data exposure	Per-user redacted caches, proof secrets in secure storage, backup exclusions, sensitive screen redaction in app switcher and log scrubbing.
Push disclosure	Generic lock-screen copy, minimal identifiers, server-controlled recipients and invalid-token removal.
Unsafe privileged SQL	Explicit actor and role checks, controlled search_path, least EXECUTE privileges; no unprotected SECURITY DEFINER function.
Missing server protection	CI fails for exposed tables without RLS, unsafe API grants, leaked secrets and unauthorized Realtime topic access.

13.2 Transport and data protection

- Require HTTPS / WSS and normal platform certificate validation; disable cleartext Android traffic and retain iOS App Transport Security. No permissive certificate bypass in production.
- Expose customer contact details only to the owner and an assigned rider during the active job. Revoke location subscriptions and contact access on completion or cancellation.
- Limit production access to named operators. Review migrations and keep role, configuration, dispatch and proof audit trails. Redact OTPs, JWTs, phone numbers and PINs from telemetry.
- Provide an accessible privacy notice and in-app account deletion request. Define retention / anonymization for order records, rider telemetry, proof attempts and device tokens before launch. Resolve active deliveries safely before deleting an account. [PROPOSED D-20; R8]

14 Non functional requirements

14.1 Performance and operational targets

Area	Target or requirement	Basis
Foreground realtime	Vendor acceptance to active customer screen p95 below 500 ms over 200 runs; no run above 1,000 ms. Excludes suspended app and push latency.	SRS 9.4 + proposed method
Quote response	p95 below 800 ms on the agreed campus mobile data profile.	PROPOSED
Native startup	Usable cached first screen in 3 s or less on the reference device; warm resume / cached screen in 1 s or less. Measure real signed release builds.	PROPOSED
Rendering	Smooth scrolling and controls targeting 60 fps on the reference device; profile dropped frames and large ticket / menu lists.	PROPOSED
Images and events	Menu images at most 60 KB where quality permits; lazy loading. App-level invalidation payload target under 1 KB; transport overhead reported separately.	PROPOSED
Availability	99.5% monthly for ordering and tracking during campus operating hours.	PROPOSED
Pilot capacity	10,000 registered users; peak 300 orders per hour; 40 online riders; 25 online vendors. Validate before launch.	PROPOSED
Supported platforms	Android and iOS; proposed test floor Android 10 and iOS 16, subject to selected framework support and campus device audit. Reconfirm store target SDK requirements at submission.	PROPOSED D-14
Accessibility	Screen reader labels, logical focus, dynamic text up to 200%, WCAG AA contrast; touch targets at least 48 dp on Android / 44 pt on iOS.	PROPOSED
Recovery	Daily database backups and point-in-time recovery where provisioned; quarterly restore drill. Exact retention and RPO / RTO set in D-20.	PROPOSED
Release observability	Crashes and ANRs, start / resume time, network errors, OTP failures, realtime latency, push receipts and stale rider telemetry.	PROPOSED

14.2 Capacity and battery validation

At 300 orders per hour, the system averages five new orders per minute at peak. Six status changes per order add about 0.5 status writes per second before fan-out. Forty riders reporting every 10 seconds add four location writes per second, or eight at a five-second active-trip interval. Push, subscriptions and reconnect bursts add work; benchmark the actual Supabase tier rather than assuming capacity.

Measure a representative one-hour rider trip with the screen on and off on the agreed Android / iOS devices. Record battery use, data use, GPS freshness and thermal effects, then set the release budget from the field trial. Do not trade reliable trip state for an unvalidated fixed background interval. [PROPOSED]

15 Verification and test plan

Retain the four inherited automated sign-off checks and run them against native applications and the real backend contracts. Add mobile readiness gates for permissions, background execution, offline recovery and signed releases. Tests use synthetic staging users and production-like release builds. [SRS 9; MOBILE]

15.1 Original sign-off checks adapted to mobile

Test	Procedure	Pass condition
T1 Authentication	Native UI test with fixed test OTP and an isolated Google token-exchange fixture; additionally run real provider sign in on signed devices. Restart and restore the app.	Phone and Google sessions work; one customer profile per identity; secure session survives restart; sign out clears it.
T2 RBAC	As customer, request VendorOrders via navigation, deep link and notification tap; call vendor_orders and unauthorized read / write APIs directly. Repeat each role.	No privileged screen or data shown; wrong-role RPC gives 403; RLS and column exposure match section 4.
T3 Pricing	Run every section 6.4 vector and boundary through Edge Function and decimal / SQL test fixtures; exercise checkout requote.	2.0 km = GHS 8.00 fee, 1.50 commission and 6.50 payout; all vectors exact; changed quote requires reconfirmation.
T4 Realtime	Use two active native clients controlled by one runner with a shared monotonic clock: vendor action start to customer rendered-state acknowledgement. Run 200 trials.	p95 below 500 ms; no run above 1,000 ms; record network, device, staging region and harness overhead.

15.2 Additional mobile readiness gates [PROPOSED]

Gate	Required scenarios	Pass condition
M1 Permissions and identity	Grant, deny and revoke GPS, camera and notifications; native Google cancel / callback; iOS login release decision.	Fallbacks work; no permission loop or unauthorized screen; configured auth clients work on signed binaries.
M2 Push and lifecycle	Foreground, background, locked device, cold launch and token rotation; duplicate / expired events and sign out.	Valid notification opens an authorized freshly read resource; no secret on lock screen; stale or duplicate hints cannot change state.
M3 Rider tracking	Physical-device active trip with background / screen lock; permission loss, battery saving and force stop.	Permitted tracking measured; stale status appears within configured threshold; no new claim from stale location; trip recovers safely.
M4 Offline and release	Drop network before and after critical server commit; kill process; relaunch; install upgrade; test min-version / compatibility behavior.	No duplicate order, claim or proof; pending state reconciles; cached reads labeled; signed builds upgrade without losing valid server state.

A mocked Google flow proves the app contract, not the provider configuration. At least one physical-device provider smoke test is required for each released platform and app. [PROPOSED]

15 Test coverage continued

15.3 Test layers

Layer	Tool or environment	Coverage
Unit	Jest and TypeScript	Distance, money, phone formatting, permitted states, idempotency state and event deduplication.
Native components	React Native Testing Library	Screen guards, loading / error / offline states, accessibility labels, text scaling and role-specific screens.
Database	pgTAP via local Supabase	RLS, grants, function actors, constraints, money splits, transition rules and concurrent claims.
Integration / native E2E	Maestro on Android emulators and iOS simulators; physical device smoke tests	Sign in → quote → order → vendor preparation → rider claim → pickup → proof.
Realtime / load	Shared-clock native harness; k6 API and WebSocket clients	T4 latency, reconnect / resume gaps, authorized topics, 100 subscribers, peak order / location traffic.
Security / release	CI checks and review	No service secrets in binaries; exposed schema policy audit, dependency / lockfile checks, safe app links and signed artifacts.
Field trial	Real campus riders and vendors	Audibility, permission recovery, background telemetry, battery, map routes and zone delivery targets.

15.4 Required negative and concurrency cases

- A customer cannot read another customer order, unrelated delivery, internal commission split or another customer proof credential.
- A vendor cannot read or modify another kitchen order or menu; a customer cannot invoke kitchen state functions.
- A rider sees no private contact before claim, cannot change another rider delivery, cannot recover a customer PIN and loses contact / tracking access after completion.
- Two riders claim one job at once: exactly one succeeds. One rider claims several jobs from two devices: serial locking preserves the two-job, 300 m and 8.0 kg limits.
- Reject a third order, combined payload above 8.0 kg, pickups 301 m apart, out-of-radius orders, unknown weights and stale rider locations.
- A profile role edit, forged app_role metadata, modified callback, guessed order id and unauthorized Realtime topic do not grant access.
- Lost response after server commit plus same-key retry creates one order / assignment / completion. A stale expected_version fails safely.
- Five wrong PINs cause lockout; an expired or reused QR cannot deliver; successful proof changes order and delivery exactly once.
- Sign out and account switching remove previous-user caches, local proof material, push association and native navigation history.

Release evidence
Record app version and build number, OS / device, backend revision, test dataset, network profile and result for T1 to T4 and M1 to M4. Background behavior and push delivery must be checked on physical devices, including at least one campus Android model with battery management enabled.

16 Environments and delivery pipeline

16.1 Environments

Environment	Mobile delivery and backend	Data
Local	Supabase CLI containers; native development build on emulator, simulator or device; fixed test OTP numbers.	Seed only
Pull request	Installable internal Android / iOS build connected to an isolated preview backend; separate app ids and OAuth callbacks.	Seed only
Staging	Signed release candidate through internal testing / TestFlight; full T1 to T4, M1 to M4 and load tests.	Synthetic
Production	Signed Android App Bundle via Google Play; iOS archive via App Store Connect, or approved private distribution if selected.	Live data

16.2 Pipeline stages

1. Lint and type-check all workspaces; run domain and native component tests. Pin dependencies, native SDK versions and lockfiles.

2. Apply migrations to a clean database; run policy, grants, constraints, concurrency and function tests. Fail for exposed tables without RLS or overly broad privileged APIs.

3. Build all selected Android / iOS binaries, checking native permissions, app identifiers, URL handlers and embedded configuration. Signing and provider credentials stay in CI secret stores.

4. Distribute an installable internal build and run native end-to-end flows against preview / staging. Include physical-device push, Google sign in and background location smoke tests.

5. On release candidate, pass T1 to T4 and M1 to M4. Prepare store metadata, privacy / data safety disclosures, permission explanations, account deletion flow and review access. [R8]

6. Approve and stage backend rollout before the mobile release. Submit signed binaries, allow for store review and use a staged rollout with monitoring and a pause / recovery procedure.

16.3 Version compatibility and recovery

Store releases do not update every device at once. Keep API and schema changes backward compatible with supported installed app versions; use expand-and-contract migrations, server feature flags and a versioned API contract. Define a minimum supported version and an understandable update-required screen. Allow active jobs to resolve safely during an upgrade.

A native permission, entitlement or SDK change needs a new signed binary. Optional over-the-air JavaScript updates must match the installed native runtime and applicable store rules. Roll back a compatible code update or pause rollout; do not assume a website-style redeployment instantly updates devices. Database recovery uses a forward corrective migration. [PROPOSED D-19]

16.4 Operations

Monitor order failures, OTP / SMS spend, crash and ANR rates, reconnect storms, vendor offline heartbeats, stale active-trip locations, push receipts and handover lockouts. Keep signed release artifacts and source maps access controlled. Fare / fleet configuration changes are audited and versioned. [PROPOSED]

17 Implementation roadmap

The inherited sequence remains identity and security, ordering, vendor fulfilment, then bicycle dispatch. Mobile engineering adds native auth, permissions, push, background tasks, real-device verification and store readiness. Estimates assume about four engineers using two-week sprints and require planning review. [PROPOSED]

Phase	Estimate	Deliverables	Exit criteria
1 Foundation	2 sprints	Native monorepo and app identifiers; signed dev builds; schema / RLS; phone and native Google sign in; secure sessions; screen guards; CI.	T1 and T2 pass on chosen platform builds; iOS login decision resolved.
2 Ordering	2 sprints	Customer app, vendor / menu screens, cart, fare engine, quote and placement; persistent idempotency; SQLite cache; native tracking screen.	T3 passes; one correct order after retries and restart.
3 Fulfilment	2 sprints	Vendor phone / tablet tickets, kitchen commands, private realtime, device registration, push outbox, sound and stock toggles.	T4 and notification scenarios pass on staging.
4 Dispatch	2 sprints	Rider feed, claim locks and fleet rules, native maps, foreground / background location, PIN / QR proof and freshness handling.	Fleet, proof and real-device location tests pass; campus field trial complete.
5 Hardening and release	1-2 sprints	Load and security review; accessibility; battery / connectivity trial; signed production artifacts; store listing, disclosures, review and operating runbook.	T1 to T4 and M1 to M4 green; release decisions closed; controlled rollout approved.

17.1 Planning dependencies

- Confirm Android / iOS launch order and whether three separate app listings are desired. Obtain organization-owned signing / store accounts and Google OAuth configuration early.
- Validate campus device mix, network conditions, drop points, bicycle graph, known item weights and operational alert escalation before the field trial.
- Choose the map provider and confirm offline licensing. Supply notification credentials and confirm the exact rider background execution and permission model.
- Decide the iOS sign-in approach before store submission. Store approval lead time is external to the sprint estimate; do not commit a launch date solely from engineering duration.

Launch gate
The inherited auth, RBAC, pricing and realtime checks remain mandatory. A mobile launch additionally requires secure session recovery, usable permission fallbacks, measured background trip behavior, safe offline retries and successfully signed and distributed builds.

18 Requirements traceability

SRS references below are inherited from the supplied v1.0 TRD. The mobile delivery change is the user instruction for this revision. The separate SRS must be updated to remove its conflicting browser / SPA wording before the specification baseline is approved.

Source	Requirement	TRD section	Verification
SRS 2	Phone OTP, Google sign in, shared auth store and customer profile trigger.	3	T1, M1
SRS 3	Role guard and database authorization; native screens replace browser routes.	4	T2, pgTAP
SRS 4	Distance-based fares and commission / payout split.	6	T3
SRS 4.1	Haversine distance calculated at checkout on the server.	6.2, 6.5	T3
SRS 5	Five core entities with controlled access and integrity.	4, 5, Appendix A	pgTAP, T2
SRS 6	8.0 kg payload, two orders within 300 m and 5.0 km radius.	8	Concurrency and fleet tests
SRS 7.1	Customer sign in, menus, ordering, fee breakdown, tracking and offline read cache.	3, 11.1, 11.6	Native E2E, M4
SRS 7.2	Kitchen tickets and alerts, preparation states and availability toggle.	7, 9.4, 11.2	Native E2E, M2
SRS 7.3	Rider radar, one-tap accept, routing and PIN / QR delivery proof.	8, 11.3, 12	Native E2E, M3
SRS 8	RLS on exposed tables with identity and role checks.	4, 13	CI audit, T2
SRS 9.1-9.4	Four automated integration sign-off checks.	15.1	T1 to T4
MOBILE	Installed applications and native navigation for all roles.	1, 2, 11, 16	Signed builds, M4
MOBILE	Native secure sessions and safe sign-in callbacks.	3, 4.1	T1, M1
MOBILE	Push, notification taps and OS lifecycle reconciliation.	9, 11.5	M2
MOBILE	Camera / location permissions and background trip tracking.	8.3, 11.4, 12	M1, M3
MOBILE	Native caches, offline failure states and process restart recovery.	10, 11.6	M4
MOBILE	Device performance, build signing and store distribution.	14, 16, 17	Release evidence, M4

18.1 Baseline changes

Browser SPA, React Router, Vite, service workers, PWA caching, IndexedDB, browser audio unlock, page-load metrics, browser support testing and website preview deployment requirements are superseded by their mobile equivalents. Backend APIs, Supabase, business pricing, fleet constraints and role separation remain part of the baseline. [MOBILE]

19 Assumptions and open decisions

The requested mobile delivery model is confirmed. The decisions below retain engineering defaults for ambiguities inherited from v1.0. They are review items rather than a reason to keep browser-specific requirements.

19.1 Inherited business and backend decisions

ID	Decision	Default used in this revision	Affects
D-1	Extended-zone distance basis	Charge GHS 2.50 per km beyond 4.0 km; 4.5 km costs GHS 13.25.	6, T3
D-2	Formula versus tier table	Tier table governs the first three zones.	6, T3
D-3	Unpriced gaps between bands	Round raw distance upward to 0.1 km before selecting the tier.	6
D-4	Payment and settlement	Out of scope; retain the fee snapshot for later settlement. Do not imply payment has been collected.	5, 10
D-5	Vendor / rider approval	New users are customers; operator-approved audited role change.	3, 4
D-6	Source of payload weights	Vendor supplies positive known item weight in grams; missing weight blocks checkout.	5, 8
D-7	Cancellation, exceptions and refunds	Customer cancels pending / accepted before preparation; atomic assignment cleanup. Post-pickup exception and refund procedure to confirm.	7, 10
D-8	Commission rounding	Half-up to two decimals; payout is the remainder. Use decimal / integer money calculations.	6
D-9	Pilot volume assumptions	10,000 users, 300 orders per peak hour, 40 riders and 25 vendors; validate against the campus.	14
D-10	Delivery radius origin	Vendor to customer drop point by Haversine; rider feed also filters pickup proximity independently.	6, 8
D-11	End-user admin role	No mobile admin role by default; protected operator process with server-side credentials only.	3, 4, 10
D-12	Assignment timing	Rider may claim accepted / preparing / ready; pickup requires ready.	7, 8

19.2 Review ownership

Product and campus operations confirm commercial rules, eligibility, cancellation and capacity. Engineering confirms data integrity, operational feasibility and implementation choices. QA records the accepted defaults in the test plan so changes to a decision update both the specification and its test vectors.

Preserved delivery constraints
The mobile revision does not change the campus fee matrix, 8.0 kg limit, two-order limit, 300 m pickup separation, 5.0 km delivery radius or the requirement for customer-authorized handover.

19 Mobile implementation decisions

19.3 Proposed mobile defaults

ID	Decision to confirm	Working default and release implication
D-13	Mobile framework and native libraries	React Native with Expo development builds and TypeScript. Native navigation, SQLite, secure storage, camera and location modules; pin a compatible supported SDK / lockfile. Map library and offline licensing chosen after a spike.
D-14	Platforms and app packaging	Android and iOS; three separately signed role apps. Vendor supports phones / tablets. Proposed device floor Android 10 / iOS 16 is subject to SDK support and campus audit; confirm launch order.
D-15	Identity, authorization and API pattern	Native Google plus phone OTP; secure session adapter, risk-based OTP controls, audited role approval and no automatic contact-based identity merging. Role-shaped reads, minimal private events and validated commands.
D-16	Rider background tracking and freshness	Five-second active / 10-second available foreground targets; permitted native background service during active trips. Ninety-second stale threshold; block new claims when stale. Validate permissions, OS limits and battery cost on devices.
D-17	Push, alert escalation and offline caching	Expo push worker over FCM / APNs with durable outbox and generic payloads; foreground vendor repeat chime every 20 seconds. Operations confirms acknowledgement timeout / escalation and retention of cache / telemetry buffers.
D-18	iOS login and store ownership	Organization-owned store / signing accounts. If guideline 4.8 applies, add Sign in with Apple as an equivalent login before iOS release, or document a valid exception. Phone OTP alone is not assumed to exempt Google login.
D-19	Release and compatibility policy	Signed Play / App Store builds or approved private distribution; staged rollout, backward-compatible APIs and minimum app version policy. OTA code updates only with native-runtime compatibility and store compliance.
D-20	Non-functional targets and data lifecycle	Performance, capacity, battery and reliability targets in section 14 need acceptance. Operations sets RPO / RTO, retention, deletion, audit and recovery rules. Mobile tests M1 to M4 are additional proposed release gates.
D-21	Meaning of multi-tenant	Single-campus pilot with multiple vendor businesses. If more than one campus launches, add campus_id / membership throughout data, RLS, pricing, device scope and realtime before release; test cross-campus isolation.

19.4 Items to close before production

Confirm packaging and platform support, iOS authentication, distribution ownership, permission disclosures, vendor missed-order escalation, exact map provider, approved recovery / deletion policies and whether campus-level tenancy is required. These are recorded decisions for delivery planning; the user-facing application remains mobile in every case.

Appendix A Reference database schema

Illustrative DDL for the retained core entities and proposed mobile additions. It omits complete command bodies, audit tables, quote / idempotency stores and production policies. Do not deploy it alone; the reviewed migration must enforce sections 4, 7, 8, 12 and 13.

A.1 Identity, catalog and orders

```
create schema if not exists private;
revoke all on schema private from public, anon, authenticated;
create type public.user_role as enum ('customer','vendor','rider');
create type public.order_status as enum
 ('pending','accepted','preparing','ready','out_for_delivery',
  'delivered','cancelled','rejected');
create type public.delivery_status as enum
 ('assigned','picked_up','delivered');

create table public.profiles (
 id uuid primary key references auth.users(id) on delete cascade,
 role public.user_role not null default 'customer',
 full_name text not null default '', phone text unique, email text unique,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);
create table public.vendors (
 id uuid primary key default gen_random_uuid(),
 profile_id uuid not null unique references public.profiles(id),
 name text not null, lat double precision not null check(lat between -90 and 90),
 lng double precision not null check(lng between -180 and 180),
 is_open boolean not null default true,
 created_at timestamptz not null default now()
);
create table public.menu_items (
 id uuid primary key default gen_random_uuid(),
 vendor_id uuid not null references public.vendors(id),
 title text not null, description text,
 price numeric(10,2) not null check(price >= 0),
 weight_g int not null check(weight_g > 0),
 is_available boolean not null default true
);
create table public.drop_points (
 id uuid primary key default gen_random_uuid(), name text not null,
 lat double precision not null check(lat between -90 and 90),
 lng double precision not null check(lng between -180 and 180),
 is_active boolean not null default true
);
create table public.orders (
 id uuid primary key default gen_random_uuid(),
 customer_id uuid not null references public.profiles(id),
 vendor_id uuid not null references public.vendors(id),
 rider_id uuid references public.profiles(id),
 drop_point_id uuid not null references public.drop_points(id),
 status public.order_status not null default 'pending',
 distance_km numeric(4,1) not null check(distance_km between 0 and 5),
 items_total numeric(10,2) not null check(items_total >= 0),
 delivery_fee numeric(10,2) not null check(delivery_fee >= 0),
 commission numeric(10,2) not null check(commission >= 0),
 rider_payout numeric(10,2) not null check(rider_payout >= 0),
 total_amount numeric(10,2) not null,
 total_weight_g int not null check(total_weight_g between 1 and 8000),
 fare_version text not null, version bigint not null default 1,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now(),
 check(delivery_fee = commission + rider_payout),
 check(total_amount = items_total + delivery_fee)
);
```

Listing 6. Core reference. order security / proof fields are isolated in private storage in A.3; the base order row is not a broadly readable client API.

Appendix A Delivery and configuration

A.2 Items, delivery, fare configuration and telemetry

```
create table public.order_items (
 id uuid primary key default gen_random_uuid(),
 order_id uuid not null references public.orders(id) on delete cascade,
 menu_item_id uuid references public.menu_items(id),
 title_snapshot text not null,
 unit_price numeric(10,2) not null check(unit_price >= 0),
 qty int not null check(qty > 0),
 weight_g int not null check(weight_g > 0)
);
create table public.deliveries (
 id uuid primary key default gen_random_uuid(),
 order_id uuid not null unique references public.orders(id),
 rider_id uuid not null references public.profiles(id),
 status public.delivery_status not null default 'assigned',
 batch_id uuid, assigned_at timestamptz not null default now(),
 picked_up_at timestamptz, delivered_at timestamptz,
 version bigint not null default 1
);
create table public.fare_tiers (
 id bigint generated always as identity primary key,
 config_version text not null, zone text not null,
 min_km_exclusive numeric(3,1), max_km numeric(3,1) not null,
 base_fee numeric(10,2) not null,
 commission_mode text not null check(commission_mode in ('flat','percent')),
 commission_value numeric(10,4) not null,
 extension_from_km numeric(3,1),
 per_km_rate numeric(10,2) not null default 0,
 sla_min_low int, sla_min_high int,
 effective_at timestamptz not null default now(),
 is_active boolean not null default true,
 unique(config_version, max_km)
);
create table public.rider_locations (
 rider_id uuid primary key references public.profiles(id),
 lat double precision not null check(lat between -90 and 90),
 lng double precision not null check(lng between -180 and 180),
 accuracy_m numeric not null check(accuracy_m >= 0),
 observed_at timestamptz not null,
 received_at timestamptz not null default now(),
 is_online boolean not null default false,
 heartbeat_at timestamptz not null default now()
);
create index orders_vendor_queue
 on public.orders(vendor_id,status,created_at desc);
create index orders_customer_history
 on public.orders(customer_id,created_at desc);
create index orders_unclaimed
 on public.orders(created_at)
 where rider_id is null and status in ('accepted','preparing','ready');
create index deliveries_rider_active
 on public.deliveries(rider_id,status);
```

Listing 7. Configuration rows must be validated for continuous non-overlapping bands and seeded from section 6.1. For the extended row use base_fee 12.00, extension_from_km 4.0, per_km_rate 2.50 and percent commission 0.15.

Integrity belongs in commands and migrations
Add server-maintained updated_at / version triggers and immutable snapshot controls. Enforce actor roles, rider / order assignment consistency, per-rider claim locks, QR / PIN attempt limits, and safe cancellation cleanup in the validated command layer. Direct client UPDATE on business tables remains revoked.

Appendix A Native data and access wiring

A.3 Device registration, proof and notification outbox

```
create table public.devices (
 id uuid primary key default gen_random_uuid(),
 user_id uuid not null references public.profiles(id) on delete cascade,
 installation_id uuid not null, app_role public.user_role not null,
 platform text not null check(platform in ('android','ios')),
 environment text not null, push_token text not null unique,
 enabled boolean not null default true,
 last_seen_at timestamptz not null default now(),
 unique(installation_id,app_role,environment)
);
create table private.handover_secrets (
 order_id uuid primary key references public.orders(id),
 pin_verifier text not null, qr_nonce_hash text,
 expires_at timestamptz, failed_attempts int not null default 0,
 locked_until timestamptz, consumed_at timestamptz
);
create table private.notification_outbox (
 id uuid primary key default gen_random_uuid(), event_id uuid not null,
 recipient_id uuid not null references public.profiles(id),
 event_type text not null, payload jsonb not null,
 status text not null default 'pending', attempts int not null default 0,
 next_attempt_at timestamptz not null default now(),
 created_at timestamptz not null default now(),
 unique(event_id,recipient_id,event_type)
);
create index outbox_pending on private.notification_outbox(next_attempt_at)
 where status = 'pending';
```

A.4 RLS, grants and profile creation

```
-- Repeat for every exposed table, including all mobile additions.
alter table public.devices enable row level security;
create policy devices_select_own on public.devices
 for select to authenticated using(user_id = (select auth.uid()));
revoke all on public.devices from anon, authenticated;
-- Registration / revocation goes through validated server endpoints.
-- Safe direct read access is opt-in; policies alone grant no table access.
grant select on public.devices to authenticated;

create function private.handle_new_user() returns trigger
 language plpgsql security definer set search_path = '' as $$
begin
 insert into public.profiles(id,role,phone,email,full_name)
 values(new.id,'customer',new.phone,new.email,
        coalesce(new.raw_user_meta_data->>'full_name',''));
 return new;
end $$;
revoke all on function private.handle_new_user() from public;
create trigger on_auth_user_created after insert on auth.users
 for each row execute function private.handle_new_user();
```

Listing 8. Profile metadata may supply a display name, never a role. Enable RLS on every other exposed table; grant only reviewed read columns / APIs. Business RPCs explicitly validate identity, role, ownership and account state.

Private tables require explicit privileged service grants for their worker paths. Revoke PUBLIC EXECUTE on privileged functions, narrowly grant approved caller roles, and test unauthenticated access. Production migrations also supply full policies, audit / idempotency stores, channel authorization and retention jobs. [R9]

Appendix B Technical reference sources

The supplied Toade3 TRD v1.0 is the source for the inherited business requirements and SRS references. The links below support the proposed mobile implementation and platform-specific constraints. Consulted 18 October 2026; recheck platform rules and compatible SDK versions before implementation and submission.

Ref	Official source and purpose
R1	**Supabase Auth with React Native** Session persistence, AppState and token refresh. supabase.com/docs/guides/auth/quickstarts/react-native
R2	**Expo Location** Native location configuration, permissions and background limits. docs.expo.dev/versions/latest/sdk/location/
R3	**Expo Push Notifications overview** Server delivery through FCM and APNs. docs.expo.dev/push-notifications/overview/
R4	**Expo Push Notifications FAQ** Delivery expectations, receipts and troubleshooting. docs.expo.dev/push-notifications/faq/
R5	**Expo SecureStore** Platform-backed secure storage; validate payload size and lifecycle. docs.expo.dev/versions/latest/sdk/securestore/
R6	**Supabase Native Mobile Deep Linking** Mobile OAuth callback and app scheme configuration. supabase.com/docs/guides/auth/native-mobile-deep-linking
R7	**Android Developers background location** Platform background execution and access constraints. developer.android.com/develop/sensors-and-location/location/background
R8	**Apple App Review Guidelines** Login Services 4.8; notifications, privacy and account deletion. developer.apple.com/app-store/review/guidelines/
R9	**Supabase changelog and current API exposure behavior** Review relevant backend changes and explicit table grants. supabase.com/changelog
R10	**Expo SQLite** Native persistent cache and database integration. docs.expo.dev/versions/latest/sdk/sqlite/

These sources document platform behavior. React Native / Expo, app packaging, OS support floors, additional tests and operating thresholds remain Toade3 engineering proposals as identified in section 19.